Exercício: o que cada função faz?

Estrutura de Dados – Árvore Binária de Busca

Aluno(a): _____ Data: ___/___/_____

arvore.h

```c
#ifndef ARVORE_H
#define ARVORE_H

typedef struct No No;

No *operacaoA(No *no, int v);
No *operacaoB(No *no, int v);
int operacaoC(No *no, int v);
void operacaoD(No *no);

void percursoTipo1(No *no);
void percursoTipo2(No *no);
void percursoTipo3(No *no);

int funcao1(No *no);
int funcao2(No *no);
int funcao3(No *no);
int funcao4(No *no);
int funcao5(No *no);
int funcao6(No *no);

#endif
```

main.c

```c
#include <stdio.h>
#include "arvore.h"

int main(void) {
    No *raiz = NULL;

    printf("F1: %d\n", funcao1(raiz));

    raiz = operacaoA(raiz, 50);
    raiz = operacaoA(raiz, 30);
    raiz = operacaoA(raiz, 70);
    raiz = operacaoA(raiz, 20);
    raiz = operacaoA(raiz, 40);
    raiz = operacaoA(raiz, 60);
    raiz = operacaoA(raiz, 80);

    printf("P1: "); percursoTipo1(raiz);
    printf("\nP2: "); percursoTipo2(raiz);
    printf("\nP3: "); percursoTipo3(raiz);

    printf("\nF2: %d", funcao2(raiz));
    printf("\nF3: %d", funcao3(raiz));
    printf("\nF4: %d", funcao4(raiz));
    printf("\nF5: %d", funcao5(raiz));
    printf("\nF6: %d", funcao6(raiz));
    printf("\nC(40): %d", operacaoC(raiz, 40));
    printf("\nC(99): %d", operacaoC(raiz, 99));

    raiz = operacaoB(raiz, 50);
    printf("\nDepois de B(50): ");
    percursoTipo2(raiz);
    printf("\n");

    operacaoD(raiz);
    return 0;
}
```

arvore.c (linhas 1–57)

```c
#include <stdio.h>
#include <stdlib.h>
#include "arvore.h"

struct No {
    int valor;
    No *esq;
    No *dir;
};

No *operacaoA(No *no, int v) {
    if (no == NULL) {
        no = malloc(sizeof(No));
        no->valor = v;
        no->esq = NULL;
        no->dir = NULL;
    } else if (v < no->valor)
        no->esq = operacaoA(no->esq, v);
    else
        no->dir = operacaoA(no->dir, v);
    return no;
}

No *operacaoB(No *no, int v) {
    if (no == NULL) return NULL;
    if (v < no->valor)
        no->esq = operacaoB(no->esq, v);
    else if (v > no->valor)
        no->dir = operacaoB(no->dir, v);
    else {
        No *aux;
        if (no->esq == NULL) {
            aux = no->dir;
            free(no);
            return aux;
        }
        if (no->dir == NULL) {
            aux = no->esq;
            free(no);
            return aux;
        }

        no->valor = funcao5(no->dir);
        no->dir = operacaoB(no->dir, no->valor);
    }
    return no;
}

int operacaoC(No *no, int v) {
    while (no != NULL) {
        if (v == no->valor) return 1;
        if (v < no->valor) no = no->esq;
        else no = no->dir;
    }
    return 0;
}

```

arvore.c (linhas 58–121)

```c
void percursoTipo2(No *no) {
    if (no == NULL) return;
    printf("%d ", no->valor);
    percursoTipo2(no->esq);
    percursoTipo2(no->dir);
}

void percursoTipo3(No *no) {
    if (no == NULL) return;
    percursoTipo3(no->esq);
    printf("%d ", no->valor);
    percursoTipo3(no->dir);
}

void percursoTipo1(No *no) {
    if (no == NULL) return;
    percursoTipo1(no->esq);
    percursoTipo1(no->dir);
    printf("%d ", no->valor);
}

int funcao1(No *no) {
    return no == NULL;
}

int funcao2(No *no) {
    if (no == NULL) return -1;
    int a = funcao2(no->esq);
    int b = funcao2(no->dir);
    return 1 + (a > b ? a : b);
}

int funcao3(No *no) {
    if (no == NULL) return 0;
    return 1 + funcao3(no->esq)
             + funcao3(no->dir);
}

int funcao4(No *no) {
    if (no == NULL) return 0;
    if (no->esq == NULL && no->dir == NULL)
        return 1;
    return funcao4(no->esq)
         + funcao4(no->dir);
}

int funcao5(No *no) {
    while (no->esq != NULL)
        no = no->esq;
    return no->valor;
}

int funcao6(No *no) {
    while (no->dir != NULL)
        no = no->dir;
    return no->valor;
}

void operacaoD(No *no) {
    if (no == NULL) return;
    operacaoD(no->esq);
    operacaoD(no->dir);
    free(no);
}
```

Instruções

O código da página anterior implementa uma árvore binária de busca em C, em três arquivos. As funções foram renomeadas com nomes neutros e os comentários foram retirados.

1. Leia cada função e, se precisar, simule a execução no papel.
2. Responda às questões sem compilar o código.
3. Justifique cada resposta citando as linhas ou a estrutura do código.

Questões

Questão 1. Complete a tabela: diga o que cada função faz e sugira um nome que descreva melhor sua finalidade.

Função	O que ela faz	Nome sugerido
`operacaoA`		
`operacaoB`		
`operacaoC`		
`operacaoD`		
`percursoTipo1`		
`percursoTipo2`		
`percursoTipo3`		
`funcao1`		
`funcao2`		
`funcao3`		
`funcao4`		
`funcao5`		
`funcao6`		

Questão 2. Desenhe a árvore formada após as sete chamadas de `operacaoA` no `main`.

Questão 3. Escreva a saída completa do programa, linha por linha.

Questão 4. Dos três percursos, qual imprime os valores em ordem crescente? Explique por que isso acontece em uma árvore binária de busca.

Questão 5. Em `operacaoB`, existem três situações quando o valor `v` é encontrado. Descreva cada uma e explique por que a função chama `funcao5(no->dir)` na terceira situação.

Questão 6. `operacaoD` visita os nós na mesma ordem de um dos percursos. Qual? Por que essa ordem é obrigatória para a função funcionar corretamente?

Questão 7. O que `funcao2` retorna para uma árvore vazia e para uma árvore com apenas um nó? Qual a diferença entre `funcao2` e `funcao3`?

Questão 8. As funções `funcao5` e `funcao6` falham em um caso específico. Qual é esse caso e o que acontece?

Questão 9. Uma nova árvore recebe, nesta ordem, `operacaoA` com 10, 20, 30 e 40. Desenhe a árvore e informe os valores de `funcao2` e `funcao4`. O que esse resultado revela sobre a estrutura?